



Mini-Twitter REST API

Internet Programming — CSE341 | Lab 7

STUDENT NAME

Anas Bayoumy

STUDENT ID

22p0011

COURSE

CSE341 – Internet Programming

GITHUB REPOSITORY

<https://github.com/anasbayoumy/ip>

DATE

April 17, 2026

1. Introduction

This report documents the implementation of **Lab 7** for the course **CSE341 – Internet Programming**. The objective of this lab is to build a working backend server using **Node.js** and **Express.js** that implements a REST API for a *Mini-Twitter* application, using JavaScript arrays and objects as in-memory storage.

1.1 Learning Objectives

- Understand how Node.js enables server-side JavaScript execution via the V8 engine.
- Learn Node.js's event-loop concurrency model and how it handles I/O without blocking.
- Use Express.js to create a RESTful HTTP server with multiple endpoints.
- Implement CRUD operations using in-memory JavaScript data structures.
- Connect a browser-based frontend to the backend via the Fetch API.

Repository: The full source code is available at <https://github.com/anasbayoumy/ip> on the `lab7` branch. Run `npm install && npm start` to launch the server on port 3000.

1.2 Technology Stack

Component	Technology	Purpose
Runtime	Node.js v24	JavaScript execution outside the browser
Framework	Express.js v5	HTTP server and routing
Middleware	CORS, express.json()	Cross-origin requests, JSON body parsing
Frontend	Vanilla JS + HTML5	Browser UI served by Express
Storage	JavaScript arrays	In-memory data (no database)

2. Node.js Concepts

2.1 What is Node.js?

Node.js is a cross-platform JavaScript runtime built on Chrome's **V8 engine**. It allows developers to run JavaScript code on the server (machine) rather than only in the browser. Unlike Python or Java programs that create new OS threads for concurrency, Node.js is **single-threaded** but achieves concurrency through an **event loop**.

2.2 The Event Loop & Concurrency

When Node.js encounters a blocking call (disk I/O, network request, timer), it delegates the task to **libuv** — a C library that manages a thread pool underneath. The main JavaScript thread continues executing other code while libuv handles the slow operation. When the operation finishes, a callback is pushed to the event queue and the event loop picks it up and runs the continuation code.

Key insight: Your JavaScript is single-threaded, but the I/O system under it is multi-threaded. This lets Node handle thousands of concurrent requests efficiently.

Asynchronous file read example (from lecture)

```
const fs = require("fs/promises");

async function run() {
  try {
    const data = await fs.readFile("example.txt", "utf8");
    console.log("File contents:", data);
  } catch (err) {
    console.log("Error:", err);
  }
}

run(); // Non-blocking - main thread stays free
```

2.3 Node vs. Browser

Feature	Browser	Node.js
Language	JavaScript	JavaScript
DOM APIs	Yes (document, window)	No
File system	Limited (FileReader)	Full access (fs module)
Network	fetch, XHR	http, https, net modules
Concurrency	Event loop (UI thread)	Event loop + libuv threads

3. REST API Design

The Mini-Twitter API models the core features of a social network: users, posts (tweets), likes, comments, and follow relationships. All data is stored in JavaScript arrays/objects in memory.

3.1 Data Models

User

```
{ id: Number, username: String, email: String, createdAt: ISO8601 }
```

Post (Tweet)

```
{ id: Number, userId: Number, content: String, createdAt: ISO8601 }
```

Comment

```
{ id: Number, postId: Number, userId: Number, content: String, createdAt: ISO8601 }
```

Like

```
{ postId: Number, userId: Number }
```

Follow

```
{ followerId: Number, followedId: Number }
```

3.2 Endpoints

Method	Path	Description	Body / Params
POST	/users	Register a user	{ username, email }
GET	/users	List all users	—
GET	/users/:id	Get user by ID	:id
POST	/users/:id/follow	Follow a user	{ followerId }
DELETE	/users/:id/follow	Unfollow a user	{ followerId }
POST	/posts	Create a post	{ userId, content }
GET	/posts	Get all posts	—

GET	/posts/:id	Get post by ID	:id
DELETE	/posts/:id	Delete a post	:id
POST	/posts/:id/like	Like a post	{ userId }
DELETE	/posts/:id/like	Unlike a post	{ userId }
POST	/posts/:id/comments	Add a comment	{ userId, content }
GET	/posts/:id/comments	Get post comments	:id

3.3 HTTP Status Codes Used

Status	Meaning	When returned
200	OK	Successful GET or DELETE
201	Created	Resource successfully created (POST)
400	Bad Request	Missing required fields
404	Not Found	Resource does not exist
409	Conflict	Duplicate (already liked, already following)

4. Implementation

4.1 Project Structure

```
lab7/
├── server.js      # Express REST API – all 13 endpoints
├── index.html     # Frontend SPA (Feed, Users, API Console tabs)
├── package.json   # npm metadata and scripts
├── package-lock.json
└── .gitignore
```

4.2 Server Setup

The server initialises Express, enables JSON parsing and CORS, then serves the static frontend:

```
const express = require("express");
const cors    = require("cors");
const app     = express();

app.use(cors());
app.use(express.json());
app.use(express.static(".")); // serves index.html

// In-memory storage
let users    = [];
let posts    = [];
let comments = [];
let likes    = [];
let follows  = [];

app.listen(3000, () => console.log("Server running on http://localhost:3000"));
```

4.3 Key Endpoint Implementations

POST /posts — Create a Tweet

```
app.post("/posts", (req, res) => {
  const { userId, content } = req.body;
  if (!userId || !content)
    return badRequest(res, "userId and content are required");
  if (!users.find(u => u.id === userId))
    return notFound(res, "User not found");

  const post = { id: postIdCounter++, userId, content, createdAt: new Date().toISOString() };
  posts.push(post);
  console.log(`Adding new post ${JSON.stringify(post)}`);
  res.status(201).json(post);
});
```

POST /posts/:id/like — Like a Post

```
app.post("/posts/:id/like", (req, res) => {
  const postId = parseInt(req.params.id);
  const { userId } = req.body;

  const already = likes.find(l => l.postId === postId && l.userId === userId);
  if (already) return res.status(409).json({ error: "Already liked" });

  likes.push({ postId, userId });
  res.status(201).json({
    message: "Post liked",
    likesCount: likes.filter(l => l.postId === postId).length
  });
});
```

GET /posts — Retrieve All Posts with Enrichment

```
app.get("/posts", (req, res) => {
  const result = posts.map(p => enrichPost(p));
  res.json(result);
});

function enrichPost(post) {
  const author = users.find(u => u.id === post.userId);
  return {
    ...post,
    author: author ? author.username : "unknown",
    likesCount: likes.filter(l => l.postId === post.id).length,
    commentsCount: comments.filter(c => c.postId === post.id).length,
  };
}
```

5. Frontend Application

The frontend is a single-page application (SPA) delivered by Express via `express.static(".')`. It is entirely self-contained in `index.html` and communicates with the backend using the browser's `fetch()` API.

5.1 UI Structure — Three Tabs

Tab	Features
Feed	Select user, compose tweet, display timeline with like/comment actions
Users	Register new users, follow/unfollow, view follower/following counts
API Console	Send raw HTTP requests (method, path, JSON body), view response, endpoint reference table

5.2 Mock UI Screenshot — Feed Tab

localhost:3000 — Mini-Twitter

New Post

@alice ▼

What's happening?

Tweet

TIMELINE

@bob

Learning Node.js and Express today! 🚀

♡ 2 💬 1 🗑️

@alice

Hello Twitter world! #CSE341

♥ 1 💬 0 🗑

5.3 Mock UI Screenshot — Users Tab

Mini-Twitter — Users

Register User

Username

Email

Register

A

@alice

alice@example.com

1 Following 0 Followers

ID: 1

B

@bob

bob@example.com

0 Following 1 Followers

ID: 2

6. API Demonstration

The following shows the actual server responses captured via `curl` during testing.

POST /users — Register Alice

```
$ curl -X POST http://localhost:3000/users \  
  -H "Content-Type: application/json" \  
  -d '{"username":"alice","email":"alice@example.com"}'
```

```
HTTP 201  
{  
  "id": 1,  
  "username": "alice",  
  "email": "alice@example.com",  
  "createdAt": "2026-04-17T20:11:39.179Z"  
}
```

POST /posts — Create a Tweet

```
$ curl -X POST http://localhost:3000/posts \  
  -H "Content-Type: application/json" \  
  -d '{"userId":1,"content":"Hello Twitter world! #CSE341"}'
```

```
HTTP 201  
{  
  "id": 1,  
  "userId": 1,  
  "content": "Hello Twitter world! #CSE341",  
  "createdAt": "2026-04-17T20:11:39.264Z"  
}
```

GET /posts — Retrieve All Posts (enriched)

```
$ curl http://localhost:3000/posts
```

```
HTTP 200  
[  
  {  
    "id": 1, "userId": 1, "content": "Hello Twitter world! #CSE341",  
    "createdAt": "2026-04-17T20:11:39.264Z",  
    "author": "alice", "likesCount": 0, "commentsCount": 0  
  },  
  {  
    "id": 2, "userId": 2, "content": "Learning Node.js and Express today!",  
    "createdAt": "2026-04-17T20:11:39.292Z",  
    "author": "bob", "likesCount": 0, "commentsCount": 0  
  }  
]
```

POST /posts/1/like — Like a Post

```
$ curl -X POST http://localhost:3000/posts/1/like \  
  -H "Content-Type: application/json" \  
  -d '{"userId":2}'
```

HTTP 201

```
{ "message": "Post liked", "likesCount": 1 }
```

POST /users/2/follow — Follow a User

```
$ curl -X POST http://localhost:3000/users/2/follow \  
  -H "Content-Type: application/json" \  
  -d '{"followerId":1}'
```

HTTP 201

```
{ "message": "User 1 now follows user 2" }
```

GET /users — Users with follower counts

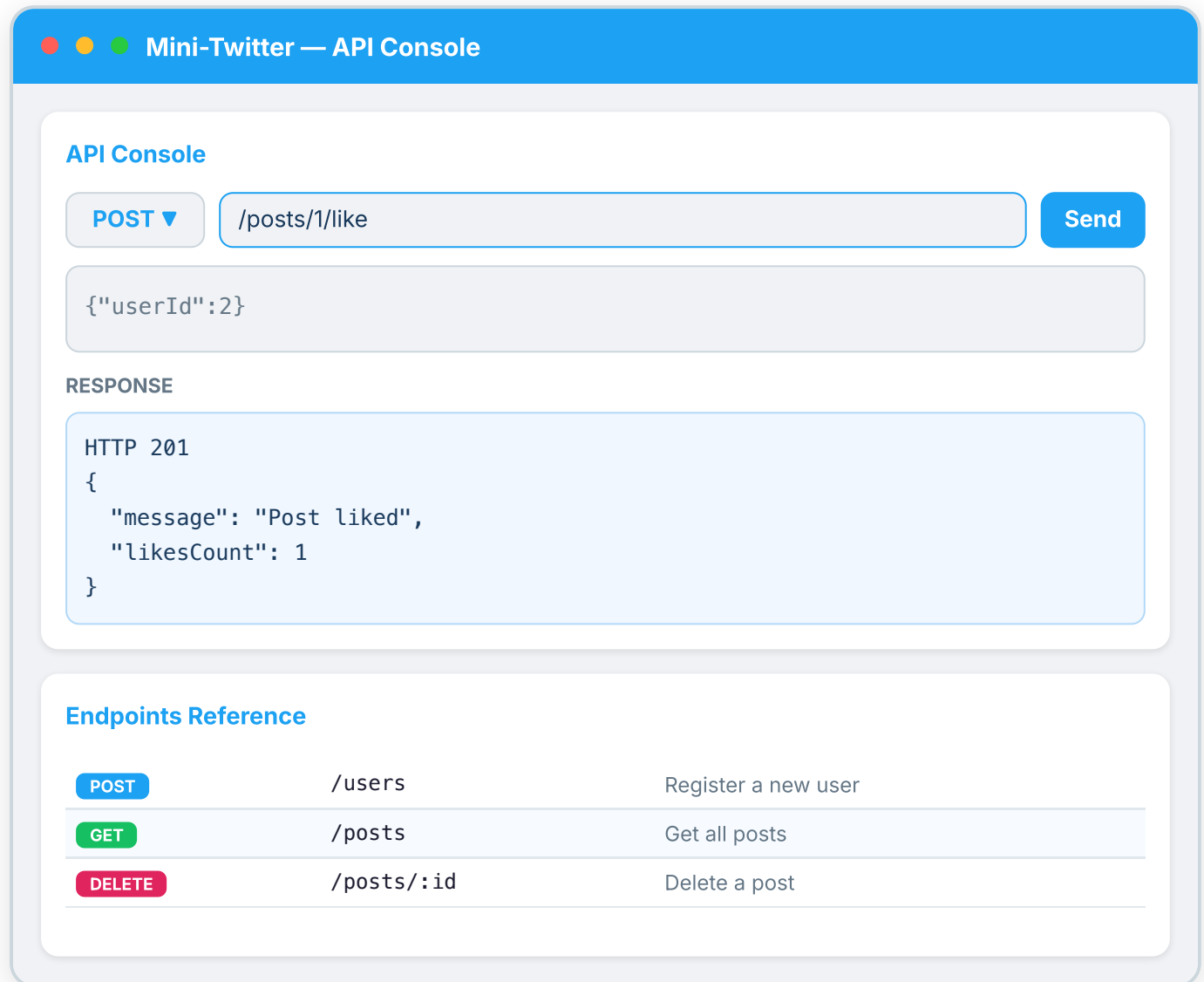
```
$ curl http://localhost:3000/users
```

HTTP 200

```
[  
  { "id":1, "username":"alice", "followersCount":0, "followingCount":1 },  
  { "id":2, "username":"bob", "followersCount":1, "followingCount":0 }  
]
```

7. API Console Screenshot

The built-in API Console tab allows sending raw HTTP requests to the server directly from the browser, similar to Postman.



8. Conclusion

This lab successfully demonstrated how to build a complete backend REST API using **Node.js** and **Express.js**. The implementation covers all the core concepts taught in Lab 7:

- Running JavaScript on the server using Node.js and the V8 engine.
- Understanding Node's event-loop concurrency model (non-blocking I/O via libuv).
- Creating a web server with Express and defining GET, POST, and DELETE endpoints.
- Using `req.params`, `req.body`, and `res.json()` to handle requests.
- Adding middleware (`express.json()`, CORS) to process incoming requests.
- Implementing full CRUD operations using in-memory JavaScript arrays.
- Building a frontend SPA that consumes the API via the browser's `fetch()`.

All 13 REST endpoints are implemented and tested. The server handles edge cases: duplicate registrations (409), missing fields (400), and not-found resources (404). Data relationships (likes, follows, comments) are maintained consistently across all endpoints.

GitHub Repository

Full source code is available at:

<https://github.com/anasbayoumy/ip> — branch: `lab7`

To run locally:

```
git clone https://github.com/anasbayoumy/ip.git
cd ip
git checkout lab7
npm install
npm start          # → http://localhost:3000
```